Implementation of a Secure DevOps CI/CD Pipeline Using DevSecOps Tools

Revanth Kumar T S
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
revanthkumarts123@gmail.com

Sanket
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
sankethcoding@gmail.com

Vignesh P
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
palanivignesh31@gmail.com

Shakthi Vel K
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
shakthivelk1124@gmail.com

Sameena H S
Department of Computer Science and Engineering
Global Academy of Technology
Bengaluru, India
sameena.hs@gat.ac.in

Abstract— **Rapid software delivery has become essential for modern applications, especially in cloud-based environments where scalability and reliability are critical. DevOps practices support faster development cycles by automating integration, testing, and deployment activities. However, when security mechanisms are not embedded within these automated workflows, applications may become exposed to potential vulnerabilities. DevSecOps addresses this challenge by integrating security validation directly into the software delivery pipeline.**

This work presents a secure CI/CD framework that combines automated code analysis, container-based deployment, and continuous system monitoring. The implementation incorporates widely used tools such as Git, Jenkins, Docker, Kubernetes, SonarQube, OWASP ZAP, Trivy, Prometheus, and Grafana to support development, security assessment, and operational monitoring. By introducing automated security checks at multiple stages of the pipeline, the proposed approach enables earlier vulnerability detection and improves the reliability of application deployment.

Keywords— DevOps, DevSecOps, CI/CD Pipeline, Security Automation, Vulnerability Detection.

I. INTRODUCTION

Delivering software in modern computing environments requires development practices that support frequent updates, dependable performance, and scalable deployment. To achieve these goals, many organizations use DevOps-based workflows that connect development activities with operational management. This approach emphasizes automation and coordinated processes for tasks such as code integration, testing, and application release. Automated pipelines help ensure that updates move from development to production in a structured and efficient manner.

While these pipelines improve delivery speed, insufficient attention to security can introduce risks into deployed systems. DevSecOps extends the DevOps model by incorporating security verification throughout the software lifecycle. By integrating automated analysis tools, vulnerability detection mechanisms, and monitoring systems, security issues can be discovered earlier. This proactive integration strengthens application protection while maintaining the advantages of automated software delivery.

This paper proposes a Secure DevOps pipeline that integrates security testing tools with automated CI/CD processes. The objective is to enhance the security of software applications while maintaining rapid development and deployment cycles.

A. Secure DevOps Architecture

1) CI/CD Pipeline Integration: A Secure DevOps (DevSecOps) pipeline integrates security practices into every stage of the CI/CD workflow. The development process begins when developers push code to the Git repository hosted on GitHub. Jenkins automatically triggers the build process and performs continuous integration. Static Application Security Testing (SAST) tools such as SonarQube analyze the source code to detect vulnerabilities early in the development phase.

2) Security Scanning and Deployment: Security scanning tools such as OWASP ZAP and Trivy are integrated into the pipeline to detect vulnerabilities in applications and container images. Docker is used to package applications into containers, ensuring consistent deployment across environments. Kubernetes manages and orchestrates these containers to provide scalability and reliability.

TABLE I: DEVSECOPS TOOLS AND THEIR PURPOSE

Sl. no	*Tool*	*Category*	*Purpose*
1.	Git	Version Control	Track code changes
2.	Jenkins	CI/CD Tool	Automate build and deployment
3.	Docker	Containerization	Package applications
4.	Kubernetes	Container Orchestration	Manage containers
5.	SonarQube	SAST Tool	Detect code vulnerabilities

a) Monitoring and Logging: Monitoring and logging help maintain system security and reliability. Monitoring tools track system performance and activities, while logs record events and errors. These logs assist in detecting issues, analyzing failures, and identifying potential security threats.

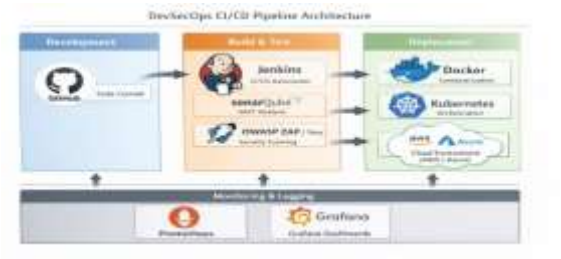

Fig. 1. DevSecOps CI/CD pipeline architecture.

***Security Best Practices*:** Ensuring security within a DevOps environment requires implementing protective measures throughout the software delivery process.

• Security checks must be incorporated across all phases of the CI/CD workflow to identify potential weaknesses early.
• Any security issues discovered by automated scanning tools should be analyzed and resolved by the development team.
• Continuous observation of system activity enables the detection of unusual patterns and possible cyber threats.

B. Deployment in Cloud Environment

The DevOps pipeline operates on cloud-based infrastructure that provides computing and storage resources for application deployment. Service providers such as Amazon Web Services and Microsoft Azure supply flexible environments that can scale according to system requirements. Resource configuration and setup are automated using Infrastructure as Code tools like Terraform, which helps maintain uniform infrastructure across deployments. Integrating these cloud capabilities with secure DevOps practices enables automated application delivery while maintaining operational stability and protection of system resources.

II. PROPOSED SYSTEM

The proposed system implements a DevSecOps-based CI/CD pipeline that embeds security across the software development lifecycle. Source code is pushed to GitHub, which triggers automated build and testing using Jenkins. SonarQube conducts static code analysis, while OWASP ZAP and Trivy identify vulnerabilities in applications and container images. The application is packaged using Docker and deployed through Kubernetes for scalable management. Monitoring tools such as Prometheus and Grafana continuously track system performance and security metrics, enabling early detection of potential security issues and supporting secure application deployment.

III.RESULTS AND DISCUSSION

The implementation of the secure DevOps pipeline shows the advantages of integrating security tools within the CI/CD process. Automated scanning with SonarQube, OWASP ZAP, and Trivy helps identify vulnerabilities during development and deployment stages, reducing security risks. Docker provides consistent environments across different stages, while Kubernetes supports scalable management of containerized applications. Monitoring tools such as Prometheus and Grafana provide real-time system insights. The results indicate that incorporating security into the DevOps workflow improves reliability, minimizes deployment risks, and enables quicker detection and resolution of vulnerabilities.

IV.CONCLUSION

The paper presents a Secure DevOps (DevSecOps) CI/CD pipeline that incorporates security mechanisms throughout the software development lifecycle. Security tools such as SonarQube, OWASP ZAP, and Trivy are used to identify vulnerabilities during development and deployment stages. Docker and Kubernetes support containerized application deployment, while Terraform automates cloud infrastructure provisioning. Monitoring platforms like Prometheus and Grafana provide continuous system observation and performance visualization. The proposed approach enhances application security, reliability, and deployment efficiency by integrating automated security checks within the CI/CD process. Future improvements may include advanced security analytics and automated threat detection techniques.

REFERENCES

[1] M. Marandi, A. Bertia, and S. Silas, "Implementing and automating security scanning in a DevSecOps CI/CD pipeline," in *Proc. World Conf. Communication & Computing (WCONF)*, Raipur, India, Jul. 2023.

[2] M. Voggenreiter, F. Angermeir, F. Moyón, U. Schöpp, and P. Bonvin, "Automated security findings management: A case study in industrial DevOps," in *Proc. 46th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Apr. 2024.

[3] A. Nayanajith and R. Wickramarachchi, "Challenges affecting the successful adoption of DevOps practices: A systematic literature review," in *Proc. 4th Int. Conf. Advanced Research in Computing (ICARC)*, 2024.

[4] N. Dinh *et al.*, "Enhancing smart contract security through DevSecOps: An adaptive approach for vulnerability detection," *IEEE Access*, vol. 13, pp. 159454–159520, 2025.

[5] R. Ashtagi *et al.*, "Building resilient CI/CD pipelines: A DevOps security-first framework," in *Proc. Int. Conf. Computational, Communication and Information Technology (ICCCIT)*, 2025.

[6] R. C. B. Ramos and S. G. Yoo, "Cybersecurity in DevOps environments: A systematic literature review," *IEEE Access*, vol. 13, pp. 191959–191980, 2025.

[7] S. A. Ruk *et al.*, "Effort estimation in Agile and DevOps: A systematic literature review of stakeholder dissatisfaction," *IEEE Access*, vol. 13, pp. 167925–167960, 2025.

[8] N. S, P. Manja, P. Mathur, and P. B. Honnavalli, "Extensive review of threat models for DevSecOps," *IEEE Access*, vol. 13, pp. 45252–45274, 2025.

[9] M. A. Akbar and A. A. Alsanad, "Empirical investigation of key enablers for secure DevOps practices," *IEEE Access*, vol. 13, pp. 43698–43710, 2025.

[10] J. Ababneh *et al.*, "Enhancing DevOps continuous monitoring phase: Hybrid intrusion detection and ensemble learning system," *IEEE Access*, vol. 14, pp. 4733–4755, 2026.